

WPipe: Framework Declarativo de Pipelines con Rastreo Forense para Workflows ML

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Abstract & Keywords

Abstract: ML training pipelines require observability, retry logic, and resource monitoring—capabilities that enterprise orchestrators (Airflow, KubeFlow) provide but at the cost of Kubernetes dependencies and 10–50× startup overhead. We present WPipe, a lightweight, declarative pipeline framework designed for single-node GPU workstations. WPipe provides: (1) declarative step definition via a `@step` decorator with automatic metadata capture, (2) forensic execution tracking in SQLite (step timing, RAM/CPU profiles, error tracebacks), (3) conditional branching via `Condition` expressions, (4) configurable retry with exponential backoff, (5) checkpoint-based resume after interruptions, and (6) real-time resource monitoring via psutil. Evaluated on the 14-step YOLO post-training pipeline, WPipe reduces pipeline authoring time from 200 lines of imperative code to 35 declarative lines, captures forensic data for 100 % of step executions (vs. 0 % with ad-hoc scripts), and adds < 2 % overhead to total pipeline execution time. The framework comprises 15 modules totaling 4,200 lines of Python and is embedded directly in the executor Docker image.

Keywords: Pipeline Framework, Declarative Pipelines, ML Orchestration, Forensic Tracking, Resource Monitoring, SQLite, YOLO Training.

2. Author Information

This research was conceptualized and developed by William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect for the wisrovi-suit ecosystem (<https://github.com/wisrovi/w-cli>). Contact: wisrovi.rodriguez@gmail.com.

3. Introduction

A YOLO post-training pipeline executes 14 sequential steps: data validation, model training, XAI visualization, uncertainty quantification, LLM report generation, and more. Each step produces artifacts (JSON, images, LaTeX tables) that downstream steps consume. When step 9 fails at 3 AM, the operator needs to know: what input caused the failure? How much RAM was the step using? Can we resume from step 9 without re-running steps 1–8?

Ad-hoc Python scripts handle none of these questions. Enterprise orchestrators—Airflow [2], KubeFlow [1], Prefect [5]—answer all of them, but require Kubernetes, Docker Swarm, or a scheduler daemon. On a single RTX 4090 workstation running YOLO training, the Kubernetes control plane alone consumes 2 GB RAM and adds 14 seconds of pod scheduling latency [4].

WPipe occupies a middle ground: lightweight enough for a single GPU workstation, observant enough for production debugging. It is a Python library, not a service. It runs in-process, stores forensic data in SQLite, and adds no infrastructure dependencies.

4. Related Work

Apache Airflow [2] is the de facto standard for workflow orchestration. It provides DAG-based scheduling, retry, and monitoring via a web UI. However, it requires a metadata database (PostgreSQL), a scheduler daemon, and a web server—a minimum of 3 processes. For a single-node ML workstation, this is overkill.

Kubeflow Pipelines [1] runs ML workflows on Kubernetes. It provides containerized step isolation and experiment tracking. The Kubernetes requirement adds 10–50× startup latency compared to in-process execution [4].

Prefect [5] and **Dagster** [3] are modern orchestrators with Python-native APIs. Both offer observability and retry, but require a server component for full functionality.

WPipe differs in scope: it targets single-node GPU workstations where the overhead of a server-based orchestrator is unacceptable. It provides forensic tracking, retry, and resource monitoring without external dependencies. The trade-off is clear: WPipe does not support distributed execution, scheduling, or multi-node coordination. For those features, Airflow or Kubeflow remain appropriate.

5. Proposed Architecture / Methodology

5.1. Core Abstraction: The @step Decorator

Steps are defined declaratively:

```
@step(name="BootstrapEval", version="v1.0")
class BootstrapEvaluator:
    def __call__(self, ctx: PostTrainContext):
        # ... evaluation logic ...
        return ctx
```

The decorator captures metadata (name, version, tags) and enables automatic tracking when the step is executed within a Pipeline.

5.2. Pipeline Orchestration

```
pipe = Pipeline(tracking_db="forensics.db")
pipe.set_steps([
    CleanFolderExtra(),
    LoadYaml(),
    CheckMinIO(),
    BootstrapEvaluator(),
    LatexExporter(),
])
result = pipe.run(initial_data)
```

The Pipeline class handles sequential execution, retry logic, checkpointing, and forensic tracking.

5.3. Forensic Tracking (SQLite)

Every step execution is recorded in a SQLite database:

- **StepExecution:** id, pipeline_id, name, version, status, duration_ms, error_message.
- **ResourceMetrics:** task_name, peak_ram_mb, avg_cpu_percent, elapsed_seconds.
- **Checkpoint:** pipeline_id, step_order, status, serialized data.

5.4. Conditional Branching

```
Condition(
    expression="ctx.get('gpu_available') == True",
    branch_true=[TrainModel()],
    branch_false=[SkipTraining()],
)
```

5.5. Retry with Backoff

Each step can specify retry behavior:

```
@step(name="Train", retry_count=3,
      retry_delay=5.0,
      retry_on_exceptions=(RuntimeError,))
```

5.6. Checkpoint Resume

The `CheckpointManager` saves step outputs to SQLite after each successful step. On pipeline restart, it resumes from the last checkpoint, skipping completed steps.

6. Experimental Setup

6.1. Infrastructure

WPipe embedded in `wyoloservice2_worker` Docker image. Python 3.12, SQLite 3.40, psutil 5.9. Hardware: NVIDIA RTX 4090, 128 GB RAM.

6.2. Evaluation

Measured on the 14-step YOLO post-training pipeline:

- Pipeline authoring time (lines of code).
- Forensic data completeness (% of steps with tracked metadata).
- Execution overhead (time added by tracking).
- Checkpoint resume time (steps skipped on re-run).

7. Results & Discussion

Cuadro 1: WPipe vs. Ad-Hoc Scripts vs. Airflow (single-node)

Feature	Ad-Hoc	Airflow
Lines of Code	200+	150+
Forensic Tracking	0 %	100 %
Retry Support	Manual	Yes
Checkpoint Resume	No	Yes
Resource Monitoring	No	Limited
External Dependencies	None	PostgreSQL + Scheduler
Startup Overhead	0s	14+ s

WPipe reduces authoring from 200+ lines to 35 while providing forensic coverage comparable to Airflow. The 0.2s overhead comes from SQLite writes (50ms per step) and psutil sampling (20ms per step).

7.1. Checkpoint Resume Efficiency

For a 14-step pipeline failing at step 11, checkpoint resume skips steps 1–10 in 0.8 seconds vs. 45 seconds for full re-execution. This represents a 56× speedup for failure recovery.

7.2. Resource Monitoring Impact

Peak RAM monitoring reveals that step 9 (LLM report generation) consumes 2.8 GB due to the Open-Code subprocess—information not visible without monitoring. This enabled targeted optimization (lazy model loading), reducing peak RAM to 1.4 GB.

8. Data & Code Availability Statement

This architecture operates under a Dual Licensing Model (PolyForm Noncommercial / AGPLv3). To deploy the project and reproduce these experiments, use the https://github.com/wisrovi/wyoloservice2_production repository.

9. Broader Impact / Ethics Statement

Lightweight pipeline frameworks democratize ML operations. Not every lab has a Kubernetes cluster. WPipe brings forensic tracking and retry logic to the single-GPU workstation, making production-grade pipeline practices accessible to individual researchers and small teams.

10. Conclusion & Future Work

We presented WPipe, a declarative pipeline framework with forensic tracking for single-node ML work-

flows. It reduces authoring effort by 83 %, provides 100 % forensic coverage, and adds < 2 % overhead. Future work will add async pipeline support (already prototyped as `PipelineAsync`), distributed step execution via Redis, and a web dashboard for real-time monitoring.

11. Acknowledgments

We thank the contributors of the wisrovi-suit project for the foundational CLI and orchestration infrastructure.

Referencias

- [1] Google Cloud. Kubeflow: Ml workflows on kubernetes. 2019.
- [2] Airbnb Engineering. Apache airflow: Workflow automation platform. 2014.
- [3] Dagster Labs. Dagster: Data asset orchestration. 2019.
- [4] William Steve Rodriguez Villamizar. Industrial experience report: The invoker-executor pattern for fault isolation in distributed YOLO training. In *IEEE International Conference on Software Engineering (ICSE) – Industrial Experience Track*, 2026.
- [5] Prefect Technologies. Prefect: Modern workflow orchestration. 2018.